Project Report: Pathfinding AI Visualizer

Comparing A Search and Greedy Best-First Search (GBFS)**

1. How I Implemented the Logic

For this project, I built a visualizer in Python using Pygame to see exactly how different pathfinding algorithms "think." I used a Priority Queue for both algorithms, so the program always knows which grid block (or "node") is the smartest one to check next.

A* Search :

I set up A* to balance two things: how far it has already traveled, and how far it *thinks* it still must go. It uses the formula f(n) = g(n) + h(n).

In my code, the g_score keeps track of the actual steps taken from the start, while the heuristic (h1) guesses the remaining distance to the target. Because it calculates both, A* is careful and makes sure it never takes a bad route just because it looked good at first.

Greedy Best-First Search (GBFS):

For GBFS, I changed the logic, so it completely ignores the distance it has already traveled. It relies entirely on the heuristic, using the formula f(n) = h(n).

This algorithm is extremely impatient it just looks at its neighbors and constantly moves toward whichever block is physically closest to the end goal, not caring if it's the most efficient route overall.

2. Pros & Cons Based on My Tests

After running both algorithms through different mazes in the visualizer, I noticed some pretty clear strengths and weaknesses for each.

A Search: *

- **Pros:**

  It is perfectly reliable. No matter how complicated I made the walls, A* always found the absolute shortest, most optimal path. It never gets tricked.

- **Cons:**

  It does a lot of overthinking. Because it checks its g_score to guarantee the best path, it ends up exploring way more blocks than it needs to in wide-open spaces, which makes it a bit slower and heavier on memory.

Greedy Best-First Search:

- **Pros:**

  It is incredibly fast when there aren't many obstacles. Since it just sprints straight toward the target, it barely wastes any time exploring dead space.

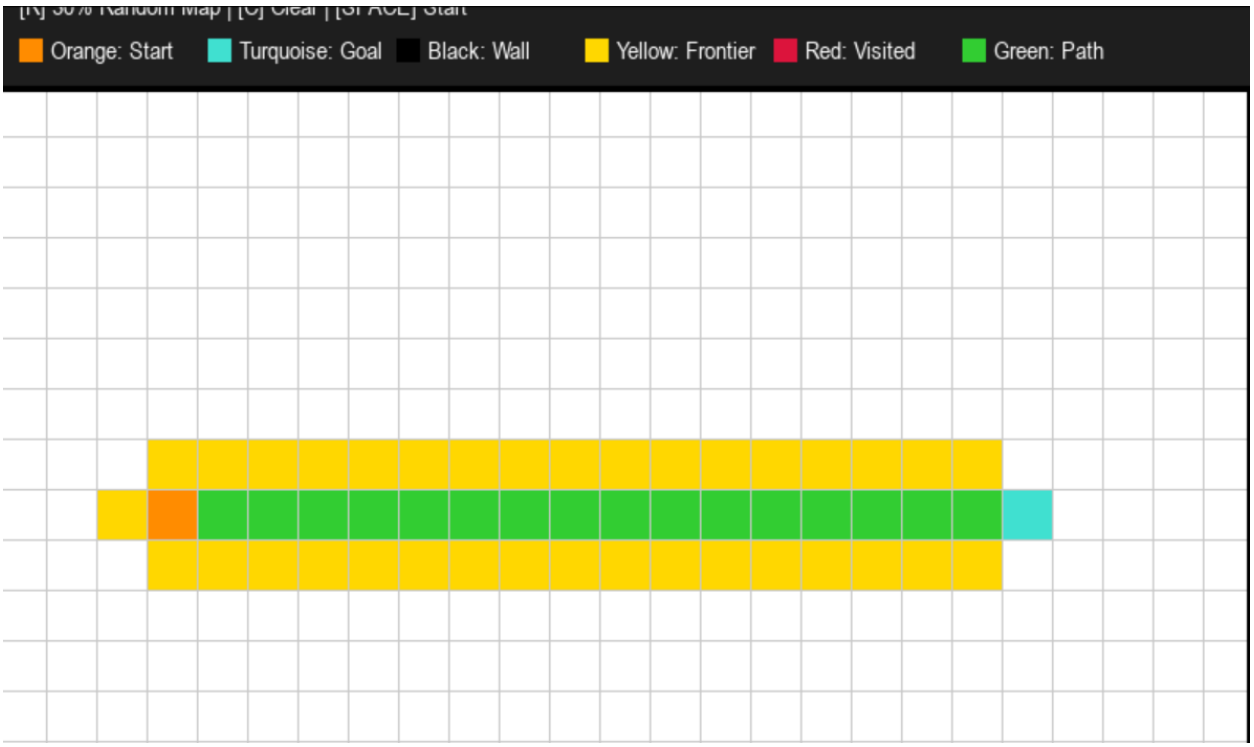
- **Cons:**

It is easily trapped and doesn't guarantee a good path. If I put a giant wall in front of it, GBFS will run straight into it and get stuck sliding up and down the wall trying to get closer to the target. The final path it draws is often jagged and way longer than it needs to be.

3. Visual Test Cases

A* Search: Best vs. Worst Case

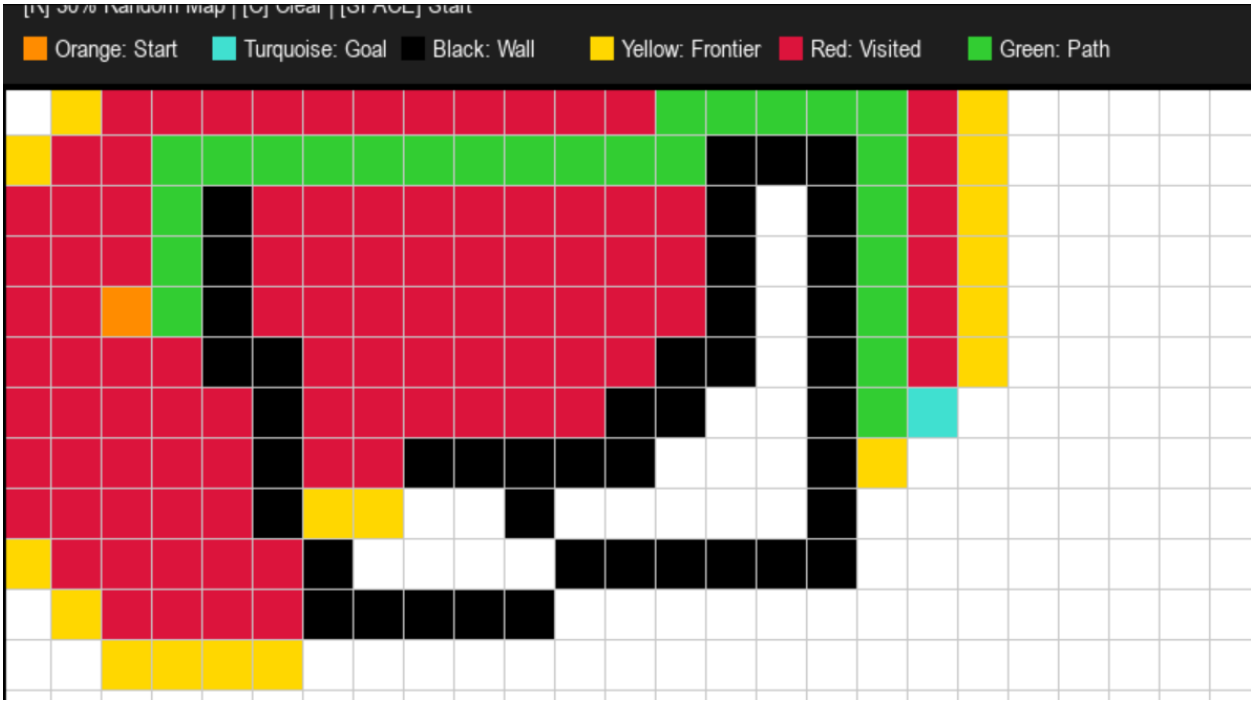
Best Case Scenario:




```
[A*] --- PATHFINDING COMPLETE ---
Path List (Row, Col Coordinates):
[(8, 4), (8, 5), (8, 6), (8, 7), (8, 8), (8, 9), (8, 10), (8, 11), (8, 12), (8, 13), (8, 14), (8, 15), (8, 16), (8, 17),
 (8, 18), (8, 19), (8, 20)]
Total Path Cost: 17 steps
Total Nodes Expanded: 52
-------------------------------------
```

In a completely open grid, A* does exactly what it's supposed to. It draws a clean, straight line to the target. The heuristic keeps it focused so it doesn't waste time checking blocks in the wrong direction.

Worst Case Scenario:




```
[A*] --- PATHFINDING COMPLETE ---
Path List (Row, Col Coordinates):
[(4, 3), (3, 3), (2, 3), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (1, 8), (1, 9), (1, 10), (1, 11), (1, 12), (1, 13), (0,
 13), (0, 14), (0, 15), (0, 16), (0, 17), (1, 17), (2, 17), (3, 17), (4, 17), (5, 17), (6, 17), (6, 18)]
Total Path Cost: 26 steps
Total Nodes Expanded: 140
-------------------------------------
```

To test its worst case, I trapped the start node inside a massive "U" shaped wall facing the wrong way. Because A* systematically checks for the absolute best path, it was forced to "flood" almost the entire inside of the trap before finally realizing it had to go backward to get around the wall.

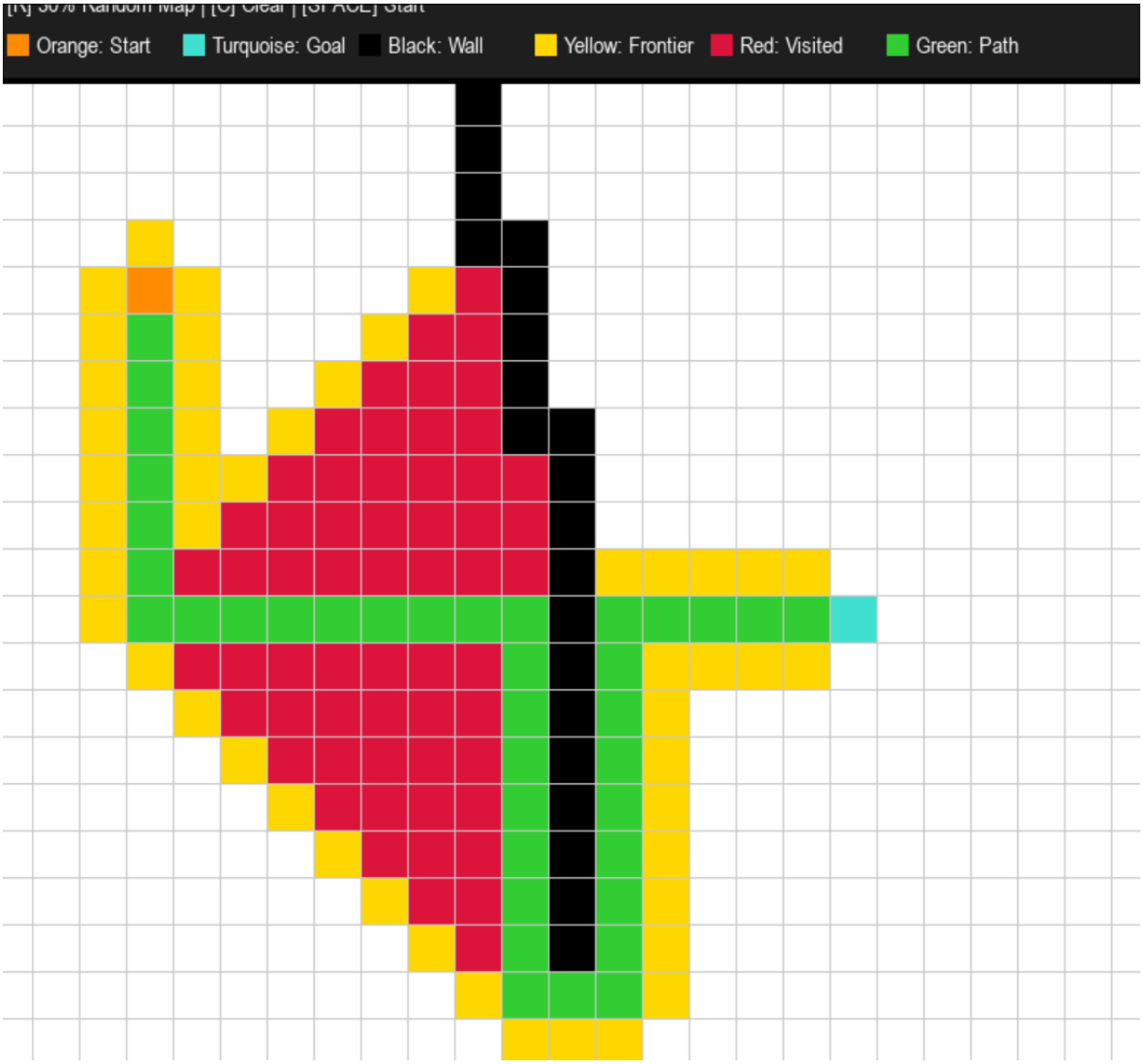
GBFS: Best vs. Worst Case

Best Case Scenario:

```
[GBFS] --- PATHFINDING COMPLETE ---
Path List (Row, Col Coordinates):
[(6, 4), (5, 4), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9), (5, 10), (5, 11), (5, 12), (5, 13), (5, 14), (5, 15), (5, 16),
(5, 17), (5, 18)]
Total Path Cost: 16 steps
Total Nodes Expanded: 48
----------------------------------------
```

When the path is clear, GBFS is brilliant. Just like A*, it sprints straight to the target, but it does so with zero hesitation and minimum memory use.

Worst Case Scenario:




```
[GBFS] --- PATHFINDING COMPLETE ---
Path List (Row, Col Coordinates):
[(5, 3), (6, 3), (7, 3), (8, 3), (9, 3), (10, 3), (11, 3), (11, 4), (11, 5), (11, 6), (11, 7), (11, 8), (11, 9), (11, 10
), (11, 11), (12, 11), (13, 11), (14, 11), (15, 11), (16, 11), (17, 11), (18, 11), (19, 11), (19, 12), (19, 13), (18, 13
), (17, 13), (16, 13), (15, 13), (14, 13), (13, 13), (12, 13), (11, 13), (11, 14), (11, 15), (11, 16), (11, 17), (11, 18
)]
Total Path Cost: 38 steps
Total Nodes Expanded: 144
----------------------------------------
```

I placed a huge, flat wall directly between the start and the end. Because GBFS is "greedy" and only wants to decrease its distance to the target, it smashed right into the center of the wall. It then awkwardly expanded up and down the entire surface of the wall before finally escaping, leaving behind an inefficient path.

Board Dynamic Size:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User1\OneDrive\Desktop\AI_Project>python dynamic_pathfinding.py
python: can't open file 'C:\\Users\\User1\\OneDrive\\Desktop\\AI_Project\\dynamic_pathfinding.py': [Errno 2]
e or directory

C:\Users\User1\OneDrive\Desktop\AI_Project>python dynamic_pathfinder.py
C:\Users\User1\AppData\Local\Programs\Python\Python313\Lib\site-packages\pygame\pkgdata.py:25: UserWarning: p
s is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources pac
ted for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  from pkg_resources import resource_stream, resource_exists
pygame 2.6.1 (SDL 2.28.4, Python 3.13.5)
Hello from the pygame community. https://www.pygame.org/contribute.html
==============================================
     AI DYNAMIC PATHFINDING INITIALIZATION
==============================================

Please enter the desired grid size (e.g., enter '35' for a 35x35 grid): 25

[+] Success! Generating a 25x25 grid map...
```